

Reading the Mold: A Constraint–Query

Decomposition of SHA-256, with a Three-Dimensional Action-Axis Localization of Irreducible Structure

Driven by Dean Kulik

June 2026

We develop and test a small, declarative *query language* for interrogating the structure of deterministic functions, built from five primitives—Direction, Boundary, Wedge (reach), Residue, and Resolution—and a **SELECT/FROM/READING** template. Rather than *computing* a function’s output, the language poses structural questions to it: what a boundary forces, how far that forcing reaches, and what residue remains once carried echo is removed. To validate the language, we calibrate every primitive against SHA-256, whose round function is a known bijection given the message schedule and therefore supplies clean ground truth. All four implemented primitives return ground truth (e.g. 896 bits of internal state forced from the digest alone, verified 28/28 words). We then apply the calibrated language to produce a three-dimensional *action-axis* decomposition of the SHA-256 state grid: lifting each (round,bit) cell by *the operation that wrote it* rather than its value. The decomposition separates a flat, redundant *conveyor echo* ($\approx 72.7\%$ of cells, exactly the six shift-coupled registers) from a 25% *action axis* confined to the two injection registers *a* and *e*, within which a 12.3% *carry buckle* carries the function’s nonlinearity and is provably carry-free at the least-significant bit. Three independently computed structural features—the conservation-law seam, the residual “escape” bits, and the carry relief—all localize to the same two register rails. We state the operative boundary explicitly and repeatedly: every result here is a property of the forward trajectory or its structural residue; *none inverts SHA-256 from the digest alone*, and preimage and collision resistance are untouched. We close with the geometric base case (the degenerate “ π -ray” triangle), an account of the readable-versus-sealed boundary that the work charts, a clearly bracketed interpretive section, and a falsifiability ledger that preserves negative results.

Introduction

Most interrogation of data is *extensional*: we store records and retrieve them. But many objects of interest are not stored—they are *intensional*, defined by rules that generate an enormous or infinite state space. A cryptographic hash, a transcendental constant, a neural network, and a system of access-control policies are all of this kind: the interesting facts are *structural*—which variables are forced, which constraints conflict, how far an influence reaches—and asking them by enumeration is hopeless.

This paper takes a deliberately narrow and testable stance toward such structure. We propose that a useful way to study a deterministic function is not to compute it, but to *query* it with a small fixed vocabulary of structural operations, and that the right way to gain confidence in such a vocabulary is to *calibrate* it against a domain where the ground truth is fully known. We use SHA-256 as that calibration domain for one reason: its compression round is a bijection given the message schedule, so it is a 1:1 deterministic object with no rounding, no approximation, and no ambiguity. If a structural query returns the right answer there—where we can check it against the actual computation—we have evidence that the query is sound, and a basis for using it where ground truth is not in hand.

The governing methodology is simple and we hold to it throughout: a claim counts only if executable code produces it, and we distinguish sharply between three statuses. **Verified** results are reproduced by code and checked against ground truth. **Open** questions are posed precisely but not settled. **Interpretive** readings are organizing metaphors that the present evidence does not establish; we confine these to a clearly labeled section and we do not let them masquerade as results. Negative results are preserved as navigational data rather than discarded.

Contributions.

(1) A five-primitive constraint-query language with a declarative template, related to existing notions in formal methods (program slicing, cone-of-influence reduction, backbone extraction, unsatisfiable cores). (2) A calibration of all four implemented primitives against SHA-256 ground truth. (3) A three-dimensional *action-axis* decomposition of the SHA-256 state grid that localizes the function's irreducible nonlinear content to a 12.3% carry relief on two register rails, and demonstrates that three independent structural features coincide on those same rails. (4) An explicit account of the boundary the work respects—no inversion from the digest, preimage resistance intact—and a falsifiability ledger.

Preliminaries: SHA-256 as a Structured Lattice

We briefly fix notation. SHA-256 processes 512-bit message blocks. Each block is expanded into a schedule of sixty-four 32-bit words $W[0], \dots, W[63]$ by the recurrence

$$W[t] = \sigma_1(W[t-2]) + W[t-7] + \sigma_0(W[t-15]) + W[t-16] \pmod{2^{32}}, \quad 16 \leq t \leq 63,$$

with $W[0..15]$ the message words and σ_0, σ_1 fixed rotate-shift mixers. The compression function maintains eight 32-bit registers (a, b, c, d, e, f, g, h) , initialized to the constants $H_0, \dots, H_7$, and iterates sixty-four rounds:

$$\begin{aligned} T_1 &= h + \Sigma_1(e) + \text{Ch}(e, f, g) + K[t] + W[t], & T_2 &= \Sigma_0(a) + \text{Maj}(a, b, c), \\ (a, b, c, d, e, f, g, h) &\leftarrow (T_1 + T_2, a, b, c, d + T_1, e, f, g). \end{aligned}$$

The round is an *Add–Rotate–Xor* (ARX) construction. Its nonlinearity comes not from randomness but from the interaction of three individually linear operations under different algebras: integer addition (carry), bitwise XOR, and rotation.

Two structural facts follow immediately from the update rule and recur throughout the paper.

Observation 1 (The register conveyor). *Only a and e receive a freshly computed value each round; the remaining six registers are exact, delayed copies: $b[t] = a[t - 1]$, $c[t] = a[t - 2]$, $d[t] = a[t - 3]$, $f[t] = e[t - 1]$, $g[t] = e[t - 2]$, $h[t] = e[t - 3]$. We verified these six identities hold with zero mismatches across all rounds for the empty message and for arbitrary test inputs.*

Observation 2 (Two rails). *The state therefore consists of two "rails"—an a -rail (a, b, c, d) and an e -rail (e, f, g, h)—each a register's freshly written value followed by three delayed copies. The entire 64×8 word grid is, up to the two injected values per round, redundant.*

We also recall the elementary projection lemma that frames the whole inversion question.

Lemma 1 (Composition cannot restore injectivity). *If $f = \pi \circ g$ where π is a non-injective projection, then no further composition on the output side recovers the information π discards. A single "still" (the digest) cannot, by post-processing alone, regenerate the "stream" (the trajectory) it was projected from.*

The trajectory of SHA-256 is reversible *given the schedule*; the digest alone is not. Everything below lives on the right side of this distinction.

The constants as cube-root edges

The SHA-256 constants are not arbitrary. The eight initial registers $H_0, \dots, H_7$ are the first thirty-two fractional bits of the square roots of the first eight primes, and the sixty-four round constants $K[0], \dots, K[63]$ are the first thirty-two fractional bits of the *cube* roots of the first sixty-four primes. We verified the derivation to high precision: for example $\sqrt{2}$ yields $H_0 = \text{0x6a09e667}$ and $\sqrt{3}$ yields $H_1 = \text{0xbb67ae85}$, while $\sqrt[3]{2}$ yields $K[0] = \text{0x428a2f98}$ and $\sqrt[3]{3}$ yields $K[1] = \text{0x71374491}$, all matching the standard exactly.

The arithmetic admits one geometric remark, kept deliberately modest. A prime is a whole number; its cube root is irrational. The constant is therefore the prime turned through its own root—the same value (it cubes back exactly, $(\sqrt[3]{2})^3 = 2$) presented in a different state (an endless fraction), with the first thirty-two bits retained. In the edge/volume language of Section 8, the cube root is the *edge* that builds the prime as a *volume*; the constant is that edge, truncated to thirty-two bits. Two properties follow that the standard relies on for diffusion: the constants are "nothing up my sleeve" (fixed by the primes, not chosen), and their bits are well mixed (an irrational fraction has no short pattern). We record the geometry as a description of the construction, not as a claim beyond it.

A Constraint-Query Language

Five primitives

We abstract the structural questions one wants to ask of a deterministic function into five composable primitives. Each has a precise referent in the formal-methods literature, indicated in parentheses.

(forward/backward slicing; pre-/post-image). Every query declares whether it traverses the dependency graph from causes to effects (*forward*: given a seed, what follows) or from effects to causes (*backward*: given

a boundary, what is forced). Choosing the wrong direction produces a false negative: a structure that reads cleanly one way can appear opaque the other.

(the pinned condition; the **SELECT** target). Every query fixes something known—a digest, a seed, a truncation—and asks what that pinning forces in the interior.

(cone of influence; program slice). The query returns *how far* the boundary’s influence extends before it goes dark or saturates—a profile of forcing-width per step, not a single value.

(unsatisfiable core; minimal correction set; the remainder after echo removal). The query removes the part of a structure explained by carried/duplicated content and reports what remains, and whether the removal was exact.

(the grain of the answer: word, bit, or shape). The query declares whether it asks for exact values, for bits, or only for *which* units are active (the support, or “shape”). Coarser resolutions read deeper than finer ones.

The template

These compose into a declarative form deliberately reminiscent of SQL, with relational algebra replaced by constraint operations:

SELECT [*forced quantity*] **FROM** [*boundary*] **READING** [*direction*] **AT** [*resolution*] **CHECKING** [*residue*].

The two primitives most prone to operator error—Direction and Resolution—thus sit in the call signature itself, so a query that points the wrong way or over-sharpens fails calibration rather than silently returning a plausible artifact.

Mapping to formal methods

Each primitive corresponds to an established operation, which is what makes the language a re-packaging of known techniques into a uniform declarative form rather than a novel solver (Table 1). The intent is the same shift relational databases underwent: from imperative traversal to a declarative query whose optimizer performs the structural reductions. Here the “optimizer” is the slicing and cone-of-influence machinery, and the “query” names which structural fact is wanted.

The five primitives and their established referents.

Primitive	Formal-methods referent	Output
Direction	forward / backward slicing; pre-/post-image	traversal sense
Boundary	the pinned assumption; SELECT target	what is fixed
Wedge / Reach	cone of influence; program slice	active sub-structure
Residue	unsatisfiable core; minimal correction set	irreducible remainder
Resolution	abstraction grain (value / bit / support)	answer granularity

Why calibrate on a 1: 1 domain

A query language for intensional structure must be debugged somewhere unforgiving before it is trusted on noisy data. SHA-256 is ideal: it is entirely deterministic, relies only on bit-vector arithmetic, and—being a bijection given the schedule—admits a fully known ground truth. If a structural query accidentally drops a necessary constraint, the check against the actual computation fails immediately and visibly. The discipline mirrors digit-extraction: just as the Bailey–Borwein–Plouffe formula proves a single hexadecimal digit of π can be isolated without computing its predecessors, the language must prove its Wedge and Direction primitives isolate the relevant slice of SHA-256 without unrolling the whole computation.

Calibration Results

We implemented the primitives as a small library and validated each against the forward-computed SHA-256 trajectory (the oracle), which the query code itself is forbidden to consult. Table 2 summarizes; details follow.

Calibration of the query primitives against SHA-256 ground truth (message "hello"). Every primitive returns truth where conviction is clean.

Primitive	Result	Status
forced_from(digest, backward)	896 bits forced, 28/28 words match truth	PASS
reach(digest, backward)	wedge [7,6,5,4,3,2,1,0], depth 7	PASS
reach(seed, forward)	onset at $W[16]$, saturates at $W[26]$	PASS
residue_after(conveyor)	six seams, zero mismatch; remainder 0	PASS

Forced state from the digest alone (backward)

Reading backward from the digest as boundary—using only the publicly known final state ($\text{state}[64] = \text{digest} - H$) and the register update relations, never the message—we recover internal state by “un-shifting” the conveyor. From $\text{state}[t + 1]$ one obtains $a[t] = b[t + 1]$, $b[t] = c[t + 1]$, $c[t] = d[t + 1]$, $e[t] = f[t + 1]$, $f[t] = g[t + 1]$, $g[t] = h[t + 1]$ for free; and since $T_2[t]$ depends only on $(a, b, c)[t]$, one further recovers $d[t]$ via $T_1[t] = a[t + 1] - T_2[t]$, $d[t] = e[t + 1] - T_1[t]$. The recovery propagates inward, losing exactly one register per round.

The result is 896 bits of internal state forced from the digest alone, with *zero* search, and all twenty-eight recovered words verified against ground truth (28/28). This recoverability is real and is also a known structural feature of ARX hashes (it is why reduced-round attacks target the final rounds); we record it because the precise shape and its boundary matter for what follows.

The backward wedge is a right triangle

The forcing width per round, reading back from round 63, is

$$[7,6,5,4,3,2,1,0],$$

a linear cascade that reaches zero after seven steps. The maximum width is 7, *not* 8: register h is permanently coupled to the message word W through the round equation and is therefore never forced from the digest alone. The cascade is thus a right triangle whose two legs both equal $7 = (\text{register count}) - 1$; the unit slope makes depth equal to width necessarily. This is the cleanest statement of where the digest's reach ends: it illuminates seven of eight registers at the boundary and drains one per round, and the single register it can never reach is exactly the one the message enters through.

The forward aperture and the {16,17} seam

Reading forward from the seed, the dependency cone of $W[t]$ —how many of the sixteen seed words can reach position t —grows as

4, 4, 7, 7, 10, 10, 13, 14, 15, 15, 16, ...

beginning with a four-seed cone at $W[16]$ and $W[17]$ and reaching full seed-mixing (all sixteen) at $W[26]$, i.e. ten rounds past the seed boundary, growing by three seeds every two rounds. The onset at {16,17} independently confirms the schedule seam: it is exactly where the expansion's dependency structure switches on, and it coincides with the round index at which the carry's algebraic degree saturates (a separate measurement). Two different reads land on the same seam.

Residue: the conveyor folds to nothing

Applying **residue_after** with the conveyor fold removes the six shift-coupled registers (each an exact delayed copy by Observation 1) and reports the remainder. Of the 512 state words, the conveyor accounts for 372 as echo, the two rails a, e account for 128, and the initial boundary accounts for 12; the remainder is exactly 0. The six conveyor seams fold with zero mismatch. The state grid is, up to the two injected values per round and a small boundary, fully explained by repetition.

Worked queries

In use, the language reads declaratively, with each question expressed by the same template and each answer checked against ground truth. "How deep does the digest illuminate the lattice, reading backward?" is **reach(digest, backward)**, returning depth 7. "Which registers does the digest force at the final round, at shape resolution?" is **forced_from(digest, backward, shape)** at round 63, returning the support vector $(a\ b\ c\ d\ e\ f\ g\ h) = (\bullet\bullet\bullet\bullet\bullet\bullet\bullet\circ)$ —seven forced, h dark. "Where is the schedule fully mixed?" is **reach(seed, forward)**, returning saturation at $W[26]$. "Does the conveyor fold leave a remainder?" is **residue_after(conveyor)**, returning 0. The same five primitives, with different boundaries and directions, express each question without bespoke code; this is the sense in which the framework functions as a query language rather than a collection of one-off scripts.

The Conservation Law and the Inversion Boundary

The backward recovery exposes a conservation relation at the final round. Writing $C[t] = T_1[t] - \Sigma_1(e[t]) - \text{Ch}(e[t], f[t], g[t]) - K[t]$, the round equation gives

$$h[t] + W[t] = C[t].$$

At round $t = 63$, the quantity $C[63]$ is computable from the digest alone (all of T_1, e, f, g at round 63 are forced), so [\[eq:cons\]](#) is a clean, checkable invariant linking the unknown register $h[63]$ to the unknown message word $W[63]$ through a known constant. We verified that [\[eq:cons\]](#) holds.

It is essential to state what this is and is not. Equation [\[eq:cons\]](#) is *one equation in two unknowns*. It constrains the pair $(h[63], W[63])$ to a line; it does not determine either. The clean, digest-only form is available for exactly one round; for earlier rounds the constant $C[t]$ itself depends on state that the backward read has not yet forced (it requires the very h values that are coupled to W), so the relation becomes contaminated by unknowns. Consequently:

Boundary. The digest-alone read forces 896 bits of internal state and yields one clean conservation constraint, but it does *not* recover the message and is *not* a preimage attack. Equation [\[eq:cons\]](#) constrains; it does not solve. Preimage and collision resistance are untouched by anything in this paper.

This is the concrete shadow of Lemma 1: the still (the digest) gives a foothold on the boundary of the stream, but walking the stream backward past the first rounds requires information the still does not carry.

The Three-Dimensional Action-Axis Decomposition

From value to action

All of the analysis so far lives on a two-dimensional grid: round (time) by bit (space), each cell holding a *value*. This plane shows what each bit *is*. It does not show what was *spent* to write it—the additions, the carries, the consumption of the message into the operations. We therefore lift the grid into a third dimension by assigning each (round,bit) cell an *action class*: the kind of operation that produced it. Concretely we distinguish

1. **flat conveyor echo** ($z = 0$): a bit that is an exact delayed copy under Observation 1;
2. **injection ridge** ($z = 1$): a freshly written bit of a or e with no carry into it;
3. **carry buckle** ($z = 2$): a freshly written bit of a or e with a carry into it, computed from the carry-in vector $(x + y) \oplus x \oplus y$ of the relevant addition;
4. **boundary cap** ($z = 3$): an early-round cell whose conveyor source does not yet exist.

Census

For the empty message, the $64 \times 256 = 16384$ -cell action grid distributes as in Table 3.

Action-class census of the SHA-256 state grid (empty message).

z	Action class	Cells	Share
0	flat conveyor echo	11904	72.7%
1	injection ridge (a, e , no carry)	2088	12.7%

<i>z</i>	Action class	Cells	Share
2	carry buckle (the relief)	2008	12.3%
3	boundary cap	384	2.3%

The relief rides on two rails, and only two

The decisive measurement is *where* the action lives. Counting carry-buckle cells per register yields

[1037, 0, 0, 0, 971, 0, 0, 0]

for registers *a, b, c, d, e, f, g, h* respectively. The carry relief appears *only* in registers *a* and *e*, and *nowhere else*—necessarily, since these are the only registers that receive a value via addition; the other six merely shift. The full action axis (classes 1 and 2) is therefore exactly the two rails *a, e*: 4096 cells, or 25% of the grid, leaving the remaining 75% as flat echo plus boundary. Within the rails the split between “ridge” (no carry) and “buckle” (carry) is nearly even (2088 vs. 2008), consistent with carries occurring in roughly half of bit positions during addition. Finally, the least-significant bit of *a* and *e* carries a buckle in 0 of 64 rounds: the LSB plane is provably carry-free—a flat “rim.”

Figure 1 renders the grid. The two vertical bands at registers *a* and *e* are the action axis; their red content is the carry buckle; everything pale is repetition.

The SHA-256 state grid lifted by action class. Rounds run top (0) to bottom (63); the horizontal axis is the 256 state bits in register order *a|b|c|d|e|f|g|h*. Pale cells are flat conveyor echo (75%); the blue and red bands at *a* and *e* are the action axis (25%); red is the carry buckle (12.3%, confined to columns *a* and *e*); the least-significant bit of each rail is carry-free. SHA-256 in this view is a mostly-flat cylinder with two buckled ridges running its length.

Three features, one structure

The result that warrants emphasis is a coincidence. Three structural features computed by entirely separate procedures all localize to the same two register rails:

- the *injection rails a, e* (Observation 2), where new values enter;
- the *residual “escape” bits*—the bits of the state grid not explained by the falling conveyor cascade—which concentrate on *a* and *e* (about 91% of the residual), at the temporal edges (rounds 0 and 63); equivalently, when the grid is wrapped into a cylinder, these sit on the two rail seams;
- the *carry buckle*, the function’s nonlinear content, confined to columns *a* and *e*.

These are not three findings but one structure seen three ways: SHA-256’s irreducible content is a thin relief on two ridges of an otherwise redundant body, carry-free at the rim.

What the decomposition does and does not establish

The decomposition *localizes* the function’s nonlinear, information-bearing content: it is the 12.3% carry buckle on two rails. It quantifies the redundancy: three-quarters of the state grid is repetition. It corrects a

loose intuition that “half” of the structure is hidden—the action is a *quarter*, and the genuinely sealing carry an *eighth*. What it does *not* do is recover the input from the relief. Localizing where the nonlinearity lives is not inverting it; the carry buckle remains nonlinear, and whether the input is recoverable from the shape of the relief is an *open* question, not one this decomposition answers. The map shows where to look; it does not claim to read what is there.

Diffusion: the Avalanche

The action-axis decomposition raises an obvious question: if three-quarters of the state grid is mere repetition and the nonlinearity is confined to two rails, how does SHA-256 achieve strong diffusion? We measured the avalanche directly. Over 2000 random single-bit input changes, the mean number of digest bits that flip is 128.1 of 256 (50.0%; range 104 to 152)—the strict avalanche behavior a secure hash requires, and our implementation reproduces the reference digests exactly. The answer is that a thin relief, iterated deeply, diffuses fully: the nonlinearity confined to the two rails, applied over sixty-four rounds and coupled through the schedule’s ten-round seed saturation (Section 4.3), suffices to carry any single-bit perturbation to a near-random half-flip of the output. Redundancy of the conveyor and concentration of nonlinearity on two rails are therefore not in tension with strong diffusion.

One clarification, which the measurement itself enforced: the function’s nonlinearity has *two* sources—integer carry and the Boolean mixers Ch and Maj. Replacing addition by XOR linearizes the schedule recurrence but not the compression, because Ch and Maj contain conjunctions and remain nonlinear; a fixed single-bit input difference through the XOR-substituted compression still produced a distinct output difference for every one of 500 random bases, confirming the residual Boolean nonlinearity. The carry buckle of Section 6 isolates the first source; the Boolean mixers, also resident on the two rails, are the second. Both live on *a* and *e*, which is why those two ridges carry the entire action axis.

The Geometric Base Case: the Degenerate π -Ray

The structural reading above has a minimal geometric exemplar, which we include because it isolates the residue/exhaust idea in its purest form. Consider the degenerate triangle with side lengths (3,1,4), satisfying $4 = 3 + 1$, so its three vertices are collinear. Its computed metrics are exact:

Quantity	Value	Quantity	Value
sides a, b, c	4,3,1 ($a = b + c$)	area K	0
angles A, B, C	$180^\circ, 0^\circ, 0^\circ (= \pi, 0, 0 \text{ rad})$	inradius	0
perimeter	8; semiperimeter 4	circumradius	$\rightarrow \infty$
medians m_a, m_b, m_c	1.0, 2.5, 3.5	difference $a - (b + c)$	0

The figure collapses to a segment of length 4. Three quantities characterize it: the difference $\bar{d} = a - (b + c) = 0$ (no gap between whole and parts), the area $\Omega = 0$ (no interior, no hidden region), and a phase coherence $\Psi = 1$ (all points aligned on one line). Yet the medians remain finite: the line still subdivides, so internal harmonic ratios persist even with the container gone.

Interpreted as a reflection J across the long side, the configuration is an eigenspace split: the line is the $+1$ eigenspace (the surviving *residue*), the perpendicular direction is the -1 eigenspace (the *exhaust*), and degeneracy drives the exhaust to zero measure. The straight angle is exactly π radians, so π appears as the angle of the collapse itself. This is the readable base case: a structure with no interior, fully determined by its boundary points, where the two distances from any interior point to the endpoints sum to the whole (the triangle inequality at equality). It is the zero-deformation limit—maximal coherence, no relief—against which the SHA-256 relief of the previous section is the deformed, information-bearing case. A diameter, in the same language, is the residue-line of a degenerate triangle; the family of right triangles inscribed on it (Thales) are its non-degenerate liftings, with the height (exhaust) restored.

The Readable–Sealed Boundary

The work charts a boundary that we believe is the genuine organizing object, and which we state carefully because it is easy to overclaim.

Some structures are *readable*: a point of interest is directly addressable from the boundary or by a closed form, without traversing the intervening state. The Bailey–Borwein–Plouffe formula is the canonical example—it extracts the n -th hexadecimal digit of π by modular exponentiation, reading rather than computing, because π 's value weighted by powers of 16 admits a position-addressable form. The degenerate π -ray is readable in the strongest sense: it has no interior to traverse. Other structures are *sealed*: the information one wants has been folded nonlinearly into the trajectory, and no boundary read recovers it. SHA-256's message is sealed in this sense—the input is consumed into the carry buckle, and the digest-alone read goes dark exactly where that buckle lives.

The three-dimensional decomposition is, in this light, a localization of the seal: SHA-256 is readable across its 75% flat echo and its injection ridges, and sealed precisely at the 12.3% carry relief. This makes the distinction concrete rather than rhetorical—we can point at the bits where reading stops.

We note a suggestive but bounded connection to quantum computation. Quantum algorithms achieve advantage exactly on readable structure: Shor's algorithm factors by reading a period off an interference pattern rather than searching, the same move as digit extraction; whereas for unstructured search the quantum speedup (Grover) is only quadratic and does not collapse the difficulty class. The readable/sealed boundary thus appears to bound quantum advantage as well. We do *not* claim this dissolves the search/verification gap; on the contrary, the bench evidence here is that the gap is real and that SHA-256 sits on its sealed side. Localizing the seal is not removing it.

Generalization: the Catalog Program

The purpose of calibrating on a 1: 1 domain is to earn the right to use the primitives where ground truth is absent. Because Direction, Boundary, Wedge, Residue, and Resolution are properties of *any* constrained structure—not of SHA-256 specifically—the natural continuation is a *catalog*: point the calibrated primitives at other deterministic structures and record where each falls on the readable–sealed boundary.

The program is falsifiable per candidate and makes specific predictions, all checkable *without recalibration*. A structure more linear than SHA-256—a linear-feedback shift register, or the schedule recurrence with the nonlinear σ mixers removed—should return a wider backward wedge (more state forced, since invertibility is higher), a larger residue echo (more of the grid explained as repetition), and a shallower carry relief (less nonlinear content). A structure that is position-addressable in closed form—the BBP digit map for π —should read as fully flat: every point reachable from the boundary, no sealed core. A structure as mixed as SHA-256, or more so, should show a deep, localized relief. If the primitives return these signatures on structures they were never tuned on, the language generalizes; if they return degenerate or incoherent output, the calibration was SHA-specific and the generalization claim fails. Both outcomes are informative, and neither requires inverting any function.

This reframes the contribution. The deliverable is not a fact about SHA-256 in isolation but a *method for locating the readable-sealed boundary instance by instance*, with SHA-256 serving as the calibration rig precisely because it is a sealed-class structure whose ground truth is fully visible in open code. The work of charting the boundary is the work of attempting to read each structure in the correct direction and at the correct resolution, and recording where the reading succeeds and where it goes dark. Each attempt—success or failure—adds a located point to the map.

Interpretation

This section is interpretive. The readings below are organizing metaphors consistent with the verified results; they are not established by them, and by their own breadth they are not presently falsifiable. We mark them as such and keep them separate from the results above.

The verified structure invites a constraint-relaxation reading. One may view the redundant conveyor as a field at equilibrium and the carry buckle as its deformation—“relief” riding on a flat ground plane (the carry-free LSB). In this reading, the input does not survive into the digest as a stored value to be read back; it is *spent* into the operations, becoming the carries and rotations—“the input became the actions.” The two rails are then the only sites of genuine action, and the difficulty of inversion is the difficulty of reconstructing the spent actions from their flattened shadow. The degenerate π -ray, with $\bar{D} = \Omega = 0$ and $\Psi = 1$, is the equilibrium limit of the same picture: full coherence, no relief, the interior vanished into the boundary, addressable from its edges alone.

A broader interpretive program in this vein treats laws and constants as residues of necessity—structures that remain after alternatives fail under recursive pressure—rather than as externally imposed rules, and seeks to trace each back to the constraint it answers. We find this a productive heuristic for choosing what to compute. We do not claim it as a result, and we explicitly flag the failure mode it must avoid: a totalizing reading that explains everything thereby predicts nothing and cannot be wrong, which by the standard of this paper is not a finding. The discipline that protects the work is the same one that produced it—executable checks, an explicit boundary, and preserved negatives.

Limitations, Falsifiability, and Preserved Negatives

We list what would break each claim and we record negative results from the same investigation.

What would falsify the results.

The conveyor identities (Observation 1) and the action census (Table 3) are direct computations; any independent SHA-256 implementation reproduces or refutes them exactly. The backward wedge, the forward saturation at $W[26]$, and the conservation law are likewise checkable against ground truth. If any of these failed to reproduce, the corresponding claim falls.

The boundary is a claim too.

We assert that none of this inverts SHA-256 from the digest. This is falsified the day a method reads the message from the digest using only the structures here; we have given direct evidence it does not (the message stays undetermined behind 896 forced bits and one two-unknown constraint), and we make no stronger claim.

Preserved negatives.

In the course of this work several appealing claims were tested and *failed*, and we keep them as data. A purported fixed carry-bit count (e.g. 1792) does not reproduce under the natural definition: the carry count is input-dependent (ranging over several thousand across inputs), so it cannot be a structural constant. A claimed lag-3 autocorrelation of π 's hexadecimal digits of $r \approx 0.758$ does not survive measurement; on 16,000 digits we find $r \approx 0.007$, consistent with zero. A circle-construction purporting to "read out" the next digits of π was found to be reverse-fit—its outputs are simple arithmetic functions of the input radii—and to contain fabricated intermediate values; the actual geometry yields different numbers. We record these because a method that only ever confirms is not being tested; the same bench that verified the results above refuted these.

The AI-consensus caveat.

Portions of this program were developed with AI assistance, and multiple language models have produced concordant accounts of the framework. We treat such concordance as *zero* evidence of correctness: identical, persuasive prompting produces a shared response that reflects the prompt, not independent discovery. Only executed code against ground truth counts.

Reproducibility

All numerical claims were produced by short Python programs implementing SHA-256 from the specification and the query primitives described in Section 3, with each primitive validated against the forward-computed trajectory it is forbidden to consult. The action-axis figure is generated directly from the empty-message computation. No external data sets are required; the calibration domain is self-contained. The query library exposes the primitives as functions (**forced_from**, **reach**, **residue_after**) with **direction** and **resolution** as explicit parameters, and a calibration routine that prints pass/fail against ground truth.

Conclusion

We have treated the structural analysis of a deterministic function as a query language, calibrated it on a domain with clean 1: 1 ground truth, and used it to produce a three-dimensional decomposition of SHA-256 that localizes the function's irreducible nonlinear content to a 12.3% carry relief confined to two register rails, carry-free at the least-significant bit, with three independent structural features coinciding on those same rails. The flat remainder—three quarters of the state—is repetition. We have stated, repeatedly and precisely, the boundary the work respects: it reads the forward trajectory and its residue; it does not invert SHA-256 from the digest; preimage and collision resistance are untouched. The geometric base case (the degenerate π -ray) supplies the readable, zero-deformation limit against which the SHA-256 relief is the deformed case, and the readable-sealed boundary—bounding, we suspect, quantum advantage as well—is the organizing object the program charts. The value of the approach is not any single number but the discipline it enforces: ask the structure directly, in the right direction and at the right resolution, check the answer where the truth is known, and keep the negatives.

Acknowledgments

Developed with AI-assisted tooling used as an execution and bookkeeping partner; all load-bearing claims were reduced to executable checks against ground truth, and concordance among language models was treated as no evidence of correctness.

99 National Institute of Standards and Technology. *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. D. Bailey, P. Borwein, S. Plouffe. On the rapid computation of various polylogarithmic constants. *Mathematics of Computation* 66(218):903–913, 1997. M. Weiser. Program slicing. *IEEE Transactions on Software Engineering* SE-10(4):352–357, 1984. F. Tip. A survey of program slicing techniques. *Journal of Programming Languages* 3(3):121–189, 1995. P. Cousot, R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *POPL*, 238–252, 1977. E. Clarke, O. Grumberg, D. Peled. *Model Checking*. MIT Press, 1999. R. Monasson et al. Determining computational complexity from characteristic phase transitions. *Nature* 400:133–137, 1999. J. Marques-Silva, M. Janota, I. Lynce. On computing backbones of propositional theories. *ECAI*, 2010. L. de Moura, N. Bjørner. Z3: An efficient SMT solver. *TACAS*, 337–340, 2008. G. 't Hooft. Dimensional reduction in quantum gravity. arXiv:gr-qc/9310026, 1993. L. Susskind. The world as a hologram. *Journal of Mathematical Physics* 36:6377–6396, 1995. R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5(3):183–191, 1961. P. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing* 26(5):1484–1509, 1997. L. Grover. A fast quantum mechanical algorithm for database search. *STOC*, 212–219, 1996. D. A. Kulik. *The π -Ray Collapse Structure: A Degenerate Triangle as an Exemplar of the Interface-Inversion Law*. QuHarmonics, The Nexus Harmonic Reality (github.com/QuHarmonics/The-Nexus-Harmonic-Reality).